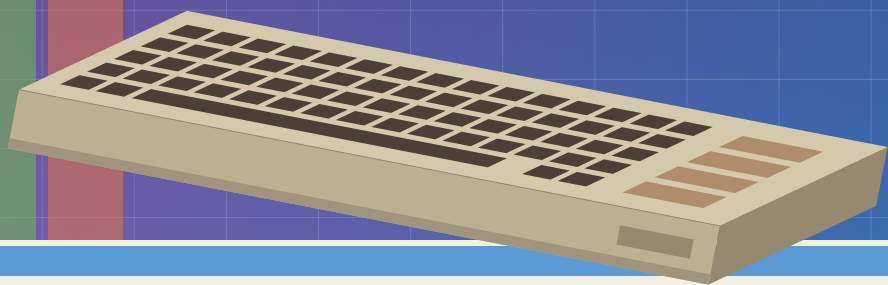


ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

RELEASE v0.4.0-14-g4ed2b58



ultimate SDK

ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

For Commodore 64 programs using the Ultimate Command Interface

First edition.

This guide documents the Ultimate SDK, a software development kit for the Commodore 64 and Ultimate hardware with the command interface.

Released under the MIT licence. Commodore, the Commodore logo and Commodore 64 are trademarks of their respective owners; this guide is not affiliated with or endorsed by them.

Contents

Introduction	v
1 GETTING STARTED	2
What The SDK Requires	2
Enable The Command Interface	2
Choose A Binding	2
Recover From Common Failures	6
2 PROGRAMMING IN BASIC	9
Two Ways To Install The Same Wedge	9
Every BASIC Keyword By Example	9
BASIC-specific Limits	13
3 PROGRAMMING IN C	16
How To Read The Examples	16
Start And Discover The Machine	17
Read SID Addresses Through Deprecated HWINFO	19
Read And Change The Live Palette	20
Use Ultimate 64 Turbo Control	20
Change And Read The Current Directory	21
Open, Read And Close A File	23
Ask What A File Is	25
Rename And Copy A File	26
Load And Save C64 Memory	26
Mount A Disk Image	27
Control The Machine And Its Drives	28
Read And Set The Clock	29
Move Bytes Through The REU	30
Inspect Network Interfaces	32
Use TCP And UDP Sockets	33
Fetch A URL In One Call	34
Build A Multi-step HTTP Request	34

Build A Request Body	36
Probe And Configure The Raw Transport	38
Execute A Raw Command With And Without Arguments	39
Read A Raw Multi-block Reply	40
Decode And Inspect Raw Status	41
4 PROGRAMMING IN ASSEMBLY	43
The Calling Convention	43
Place The SDK's Working Memory	44
Initialize, Detect And Describe The Machine	45
Read And Change The Palette	50
Use Turbo And Badline Control	51
Change And Read The Current Directory	52
Open, Seek, Read, Write And Close Files	54
Ask What A File Is	56
Rename And Copy A File	57
Load And Save C64 Memory	58
Mount A Disk Image	58
Control The Machine And Its Drives	60
Read And Set The Clock	62
Move Bytes Through The REU	64
Inspect Network Interfaces	67
Use TCP And UDP Sockets	68
Use The HTTP Client	70
Build A Request Body	71
Probe And Configure The Raw Transport	74
Raw Commands: Optional Arguments	75
Read A Raw Multi-block Reply	77
5 OTHER TOOLCHAINS	79
Build And Identify The Blob	79
Call The Stable Jump Table	80
Use The Parameter Block	81
Call The Blob From BASIC	81
Choose A Different Base Address	82
6 APPENDIX	84
Result Codes	84

INTRODUCTION

The **ULTIMATE SDK** gives a C64 program one API for the Ultimate Command Interface, the UCI, and for the two supported features that use direct hardware registers: transfers through the RAM Expansion Unit, the REU, and Ultimate 64 turbo control. The language chapters cover every public call.

The implementation is 6502 assembly. cc65 links it as a library; ca65 callers use its assembly ABI, the Application Binary Interface described in Chapter 4; BASIC reaches a banked build through a wedge; other toolchains call the same object code through a standalone jump-table binary.

How to read this

Chapter 1 covers setup and chooses a binding. Chapters 2 to 5 document the binding-specific rules and every public entry point. Each example says what feature it demonstrates; the appendix holds the result-code table.

A word about the name

The library is called the Ultimate SDK rather than the Ultimate 64 SDK because the command interface predates the Ultimate 64. Programs probe targets with `ultimate_detect()` and test the REU and turbo registers separately.

CHAPTER 1

GETTING STARTED

- What the SDK requires
- Enabling the command interface
- Choosing and checking a binding
- Recovering from common failures

WHAT THE SDK REQUIRES

The SDK runs on a C64 with Ultimate hardware whose command interface is mapped at \$DF1B–\$DF1F. It is supported on the 1541 Ultimate-II and later products.

There is no blanket minimum firmware version. Targets were added over time, so programs call `ultimate_detect()` and test the capability they need. The palette is newer than firmware 3.15; HTTP appears in 3.15.

The control target predates the palette commands, so its presence is not a palette capability test. Call a palette function and handle `ULTIMATE_ERR_NOT_SUPPORTED`; the SDK returns that result when older firmware answers that it does not know the command.

ENABLE THE COMMAND INTERFACE

Open the Ultimate configuration menu and set `COMMAND INTERFACE` to `ENABLED`. The exact key used to open that menu depends on the product and its configuration, so the SDK does not prescribe one.

With the interface disabled, the five registers are not mapped and `ultimate_init()` returns `ULTIMATE_ERR_NO_DEVICE`.

REU transfers and Ultimate 64 turbo control use their own registers, not the command interface. Each still depends on its corresponding machine setting.

CHOOSE A BINDING

BASIC V2	<code>make wedge; load src/basic/uci.prg, or mount src/basic/uci.crt</code>
cc65 C	<code>make lib; include ultimate.h and link bindings/cc65/build/ultimate.lib</code>
ca65 assembly	<code>link the same library and include bindings/asm/ultimate.inc</code>
other toolchains	<code>make blob; load the standalone binary described in Chap- ter 5</code>

BASIC check

Load and run the wedge. It installs itself, prints a banner and performs a NEW, so the BASIC program area is empty again afterwards:

```
LOAD"UCI",8
SEARCHING FOR UCI
LOADING
READY.
RUN
ULTIMATE UCI BASIC WEDGE
SDK v0.4.0-14-g4ed2b58
VIVA LA COMMODORE!
READY.
```

Now send a command. UDIR lists the Ultimate's current directory, and UERR reports how that command ended:

```
UDIR
DATA
BIG.BIN
README.TXT
READY.
PRINT UERR
0
READY.
```

The entry names depend on the machine and the medium; the printed result does not. A printed 0 is ULTIMATE_OK; 1 is ULTIMATE_ERR_NO_DEVICE. Together these two screens exercise wedge installation, a directory command and the shared result model.

C check

Build the library once before either of the two checks below:

```
make lib
```

OUTPUT

bindings/cc65/build/ultimate.lib is created.

This complete program initializes the SDK and reports any failure. Save it as `hello.c`:

```
#include <stdio.h>
#include <ultimate.h>

int main(void)
{
    uint8_t err = ultimate_init();

    if (err != ULTIMATE_OK)
        printf("ultimate: %s\n", ultimate_strerror(err));
    return err != ULTIMATE_OK;
}
```

OUTPUT

No text on success. On failure, for example:
ULTIMATE: NO ULTIMATE FOUND

Build it from the repository root:

```
cl65 -t c64 -I include -o hello.prg hello.c \
    bindings/cc65/build/ultimate.lib
```

OUTPUT

hello.prg is created with no compiler errors.

Assembly check

This is the same check in ca65 assembly, complete enough to assemble, link and run. Save it as hello.s:

```
.include "ultimate.inc"

.forceimport __STARTUP__
.export _main

CHROUT = $FFD2

.code

_main: jsr ultimate_init
      cmp #ULTIMATE_OK
      bne failed

      ldx #0
found: lda msg_found,x
      beq done
      jsr CHROUT
      inx
      bne found

done:  rts

failed: ldx #0
absent: lda msg_absent,x
      beq done
      jsr CHROUT
      inx
      bne absent

.rodata

msg_found: .byte "ultimate found", 13, 0
msg_absent: .byte "no ultimate found", 13, 0
```

Build it from the repository root:

```
cl65 -t c64 --asm-include-dir bindings/asm \
-o hello.prg hello.s bindings/cc65/build/ultimate.lib
```

OUTPUT

```
ULTIMATE FOUND
```

Three details in that listing matter. A program that leaves any of them out will not link, or will link and print nothing.

`.forceimport __STARTUP__` and the name `_main` pull in cc65's start-up code, which is the simplest way to get a running `.prg`. The SDK itself needs no C runtime; Chapter 4 covers linking without one.

`--asm-include-dir` is how `ultimate.inc` is found.

The message text is lowercase in the source and prints as capitals. ca65's C64 character map turns source `a–z` into the byte values the machine displays as `A–Z`.

The result convention is the one every entry point uses: `jsr`, then `cmp` against `ULTIMATE_OK`, then branch. Compare `A` straight away, before another instruction overwrites it.

RECOVER FROM COMMON FAILURES

No device

`ULTIMATE_ERR_NO_DEVICE` means the signature is absent. Check the Command Interface setting before assuming the hardware is missing.

An abandoned exchange

A program can stop while the firmware is holding command or reply state. `uci_abort()` releases it; `ultimate_init()` performs that recovery during startup. From BASIC, a bare UCI calls the same abort path:

```
UCI:PRINT UERR
0
READY.
```

This example demonstrates recovery only; it does not send a command.

A missing feature

Use `ultimate_detect()` for command-interface targets, `ultimate_reu_available()` for the REU, and `ultimate_turbo_available()` for turbo. Do not infer capabilities from a model name.

A short buffer

ULTIMATE_ERR_TRUNCATED means an exchange completed but not all reply bytes fitted. Output lengths report the bytes retained, not the bytes discarded.

CHAPTER 2

PROGRAMMING IN BASIC

- How the two wedge builds differ
- The twenty-three keywords
- Clear examples of each service group
- BASIC-specific limits

TWO WAYS TO INSTALL THE SAME WEDGE

Chapter 1 shows the `uci.prg` installation. Its loader copies the SDK under BASIC ROM at \$A000 and the wedge to RAM at \$C000, installs four BASIC vectors, then performs `NEW`. The normal BASIC program area remains available.

The `uci.crt` build carries the same runtime bytes in an 8K cartridge. It autostarts and its warm-start vector reinstalls the wedge after reset. While the cartridge remains mapped at \$8000-\$9FFF, BASIC's program area is 8K smaller.

Both builds contain the same wedge; only their loaders differ.

Careful: The wedge owns BASIC's tokenise, list, statement-dispatch and expression vectors and does not chain previous owners. Do not combine it with another extension that needs any of those vectors unless that combination provides an explicit compatibility layer.

EVERY BASIC KEYWORD BY EXAMPLE

The wedge exposes twenty-three keywords. This section uses every one. Services without a dedicated keyword remain available through the raw UCI statement. Output below is from a representative successful run; model names, times, directory entries and REU size depend on the machine. When a result is fixed, it appears directly after `RUN` in the same screen.

Run a command and inspect every result

```
10 UCI UCTRL,1
20 PRINT "RESULT";UERR;" DEVICE";UDEV;" BYTES";ULEN
30 PRINT "STATUS: ";UST$
40 PRINT "DATA: ";UDAT$
50 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
RESULT 0 DEVICE 0 BYTES 14
STATUS: 00,OK
DATA: CONTROL TARGET
67 79 78 84 82 79 76 32 84 65 82 71 69 84
READY.
```

`UCI UCTRL,1` asks the control target to identify itself. `UERR` is the SDK result, `UDEV` is the target's raw status number, and `ULEN` is the retained reply length. `UST$` returns the status text, `UDAT$` returns up to 255 reply bytes as a string, and `UBYTE(N)` returns one retained

byte or zero when N is past the end.

Use every target constant

```
10 PRINT UDOS1,UDOS2,UNET,UCTRL,UIEC,UHTTP
RUN
1 2 3 4 5 6
READY.
```

The printed values are 1 through 6: the two DOS instances, network, control, SoftwareIEC and HTTP targets. DOS here is the Ultimate's own disk operating system, which is what serves files from its storage. Use these names as the first argument to UCI; do not copy their numeric values into programs.

Send a raw command with and without an optional argument

```
10 UCI UDOS1,38: PRINT UDAT$
20 UCI UDOS1,38,0: PRINT UDAT$
30 UCI UCTRL,40: PRINT UDAT$
40 UCI UCTRL,40,0: PRINT UDAT$
50 IF UERR THEN PRINT "COMMAND FAILED";UDEV
```

Command 38 is `DOS_CMD_GET_TIME`. Line 10 omits its optional format byte; line 20 supplies it explicitly. Command 40 is `CTRL_CMD_GET_HWINFO`; the firmware documentation marks it deprecated. Lines 30 and 40 show its omitted and explicit model selector only to make the wire format clear. Both commands default an omitted argument to zero. A number normally contributes one byte; known wide arguments are widened automatically.

Read SID addresses through deprecated HWINFO

The SID is the C64's Sound Interface Device, its sound chip. An Ultimate 64 can present more than one, and this reports where each one is mapped.

```
5 REM LEGACY: HWINFO IS DEPRECATED
10 UCI UCTRL,40,1
20 IF UERR THEN PRINT "SID INFO FAILED";UERR:END
30 PRINT "SID COUNT";UBYTE(0)
40 FOR I=0 TO UBYTE(0)-1
50 A=1+5*I
60 PRINT "SID";I+1;UBYTE(A)+256*UBYTE(A+1)
70 NEXT
RUN
SID COUNT 4
SID 1 54272
SID 2 54272
SID 3 54272
SID 4 54272
READY.
```

The explicit selector 1 requests SID information through a command the firmware documentation marks deprecated. Firmware still implements it but may remove it, and no replacement C64-side UCI command is currently published. Byte zero is the record count. Each five-byte record starts with the primary address, least-significant byte first; line 60 reconstructs that address. Decimal 54272 is \$D400. The following two bytes are the optional secondary address and the fifth is a raw type value, not a 6581 or 8580 model identifier.

Force raw argument widths

```
10 UCI UDOS1,240,UW(4660),UL(65536)
20 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
1 240 52 18 0 0 1 0
READY.
```

Command 240 echoes the target byte, command byte and arguments. The leading 1 and 240 confirm the framing. UW(4660) sends 52,18 and UL(65536) sends 0,0,1,0: both are little-endian. These overrides are for commands newer than the wedge's shape table or deliberately raw byte layouts.

Abort an unfinished exchange

```
UCI:PRINT UERR
0
READY.
```

A bare UCI sends no command. It aborts any exchange left unfinished by a stopped program and returns the interface to idle; UERR reports whether the recovery succeeded.

Set and read turbo

```
10 UTURBO 3: IF UERR THEN PRINT "TURBO UNAVAILABLE":END
20 PRINT "SPEED INDEX";UTURBO
RUN
SPEED INDEX 3
READY.
```

The statement form sets an index; the function form reads it. Portable indices 0 through 3 select 1, 2, 3 and 4MHz. A read of 255 means the turbo registers are unavailable.

Load a PRG with and without the optional address

A PRG is a Commodore program file: two bytes giving the address it loads at, then the bytes themselves.

```
10 ULOAD "GAME.PRG": IF UERR THEN END
20 ULOAD "FONT.PRG",49152: IF UERR THEN END
```

Line 10 consumes the PRG's two-byte header and loads at the address stored there. Line 20 still consumes the header but loads the remaining bytes at 49152. Dedicated filename keywords copy at most 63 bytes and add a terminator.

Load and save raw memory

```
10 UBLOAD "FONT.BIN",49152,2048: IF UERR THEN END
20 USAVE "SCREEN.BIN",1024,1000: IF UERR THEN END
```

UBLOAD reads at most 2048 raw bytes into address 49152; it consumes no header. USAVE replaces SCREEN.BIN with 1000 bytes beginning at C64 address 1024.

List the current directory

```
UDIR
```

UDIR takes no parameters. It opens the current Ultimate directory, prints every entry and leaves UERR at zero when the normal end marker is reached.

Copy memory to and from the REU

```
10 R=UREU:PRINT "REU BANKS";R:IF R=0 THEN END
20 USTASH 1024,0,1000:IF UERR THEN END
30 PRINT CHR$(147)
40 UFETCH 1024,0,1000:IF UERR THEN END
```

UREU returns the size of the RAM expansion in 64K banks. USTASH copies the 1000-byte C64 text screen to REU address zero; UFETCH copies it back. The expansion moves the bytes itself with the processor halted, which is called direct memory access, or DMA. The REU address is 24 bits. Keep the address plus length within UREU times 65536; a DMA length of zero means 65536 bytes.

BASIC-SPECIFIC LIMITS

A statement extension cannot follow THEN directly

BASIC V2's IF path enters the ROM statement dispatcher instead of the wedge vector, so this is rejected:

```
10 IF X = 1 THEN ULOAD "GAME.PRG"  
RUN  
?SYNTAX ERROR IN 10  
READY.
```

Branch to a line containing the extension statement:

```
10 IF X = 1 THEN 100  
20 END  
100 ULOAD "GAME.PRG"
```

This demonstrates the workaround. Extension functions are unaffected, so IF UERR THEN 100 remains valid.

Reply storage is finite

The wedge retains 256 reply bytes. UDAT\$ can expose only 255 because that is BASIC's maximum string length; UBYTE(0) through UBYTE(255) can inspect the retained bytes individually. A longer raw reply sets UERR to ULTIMATE_ERR_TRUNCATED.

CHAPTER 3

PROGRAMMING IN C

- Startup, discovery and diagnostics
- Files, disk images, the clock and the REU
- Machine and drive control
- Network and HTTP services
- The complete low-level UCI API

HOW TO READ THE EXAMPLES

Every public function from `ultimate.h` and `uci.h` appears below. Each listing shows one complete operation within a program; calls share state only when the text says so. The guide build compiles every displayed call form. Unless a listing shows otherwise, it runs after including `stdio.h`, `string.h`, `ultimate.h` and `uci.h`. Result-returning calls use this pattern:

```
uint8_t err = ultimate_init();
if (err != ULTIMATE_OK)
    printf("init: %s\n", ultimate_strerror(err));
```

OUTPUT

```
No text on success. On failure, for example:
INIT: TIMEOUT
```

Output boxes show a representative successful run on an Ultimate 64 Elite. Model names, paths, directory entries, addresses and network replies vary with the machine and the files or servers used.

The C64 holds text in PETSCII, the PET Standard Code of Information Interchange. The Ultimate speaks ASCII. The two codes agree on the digits and on the uppercase letters and disagree on case, and cc65 maps lowercase source letters to the uppercase byte values the two share. Path, filename and header literals therefore appear in lowercase source even though the firmware receives uppercase text.

Zero is success. Appendix A lists the remaining results. Include `ultimate.h` for the service API; include `uci.h` only for the raw transport examples at the end of this chapter.

START AND DISCOVER THE MACHINE

```
int main(void)
{
    ultimate_capabilities caps;
    char text[64];
    uint16_t length;
    uint8_t err = ultimate_init();

    if (err != ULTIMATE_OK || !ultimate_available()) return 1;
    err = ultimate_detect(&caps);
    if (err == ULTIMATE_OK && ultimate_has_http(&caps))
        puts("http target present");

    err = ultimate_identify(UCI_TARGET_DOS1,
        text, sizeof text, &length);
    if (err == ULTIMATE_OK) printf("dos: %s\n", text);
    err = ultimate_identify(UCI_TARGET_DOS1,
        text, sizeof text, NULL);

    err = ultimate_get_model(text, sizeof text, &length);
    if (err == ULTIMATE_OK)
        printf("model received: %u bytes\n", length);
    err = ultimate_get_model(text, sizeof text, NULL);
    return err != ULTIMATE_OK;
}
```

OUTPUT

```
HTTP TARGET PRESENT
DOS: ULTIMATE-II DOS V1.2
MODEL RECEIVED: 17 BYTES
```

`ultimate_init()` performs the required startup check; `ultimate_available()` reads the resulting SDK state without touching hardware. `ultimate_detect()` fills `caps`; use the `ultimate_has_*`() macros to test targets. The paired identify and model calls show the optional final parameter: pass `&length` to receive the stored string length, or `NULL` when only the terminated string matters. The firmware documentation marks the command behind `ultimate_get_model()` deprecated; the SDK retains this existing wrapper for compatibility.

Report an error and its raw device code

```
if (err != ULTIMATE_OK) {  
    printf("%s (device %u)\n",  
        ultimate_strerror(err), ultimate_device_code());  
}
```

OUTPUT

```
TIMEOUT (DEVICE 0)
```

`ultimate_strerror()` returns stable SDK text. `ultimate_device_code()` returns the last target-specific numeric status for diagnostics; do not use that firmware-specific number as application control flow.

READ SID ADDRESSES THROUGH DEPRECATED HWINFO

The SID is the C64's sound chip, the Sound Interface Device. An Ultimate 64 can present more than one of them, and this call reports where each one is mapped.

```
ultimate_sid_info info;
uint8_t i;

err = ultimate_legacy_get_sid_info(&info);
if (err == ULTIMATE_ERR_NOT_SUPPORTED)
    puts("legacy sid query unavailable");
else if (err == ULTIMATE_OK)
    for (i = 0; i < info.count; ++i)
        printf("sid %u: $%04x type $%02x\n",
            (uint8_t)(i + 1), info.sid[i].primary_address,
            info.sid[i].type);
```

OUTPUT

```
SID 1: $D400 TYPE $83
SID 2: $D400 TYPE $83
SID 3: $D400 TYPE $85
SID 4: $D400 TYPE $85
If firmware removes the command instead:
LEGACY SID QUERY UNAVAILABLE
```

Firmware still implements this query, but its documentation marks `CTRL_CMD_GET_HWINFO` deprecated and publishes no replacement C64-side UCI command. The function is therefore named `ultimate_legacy_get_sid_info()`; always handle `ULTIMATE_ERR_NOT_SUPPORTED` in case the command is removed.

The call fetches the count and every configured address in one operation. Each record also has a `secondary_address`; zero means there is no second mapping. The type byte is retained exactly as the firmware returns it. It distinguishes current SID implementations and mapping modes, not 6581 from 8580, so do not use it as a chip model. Check the result before reading the records.

READ AND CHANGE THE LIVE PALETTE

```
uint8_t palette[ULTIMATE_PALETTE_BYTES];
uint8_t err;

err = ultimate_palette_get(palette);
if (err == ULTIMATE_OK) {
    palette[0] = 0;
    palette[1] = 0;
    palette[2] = 0;
    err = ultimate_palette_set(palette);
}

err = ultimate_palette_set_color(6, 255, 0, 0);
err = ultimate_palette_reset();
```

OUTPUT

```
SET COLOUR 6 TO RED: ULTIMATE_OK
RESTORE BUILT-IN PALETTE: ULTIMATE_OK
```

`ultimate_palette_get()` reads all sixteen RGB triples and `ultimate_palette_set()` writes all of them. The single-colour call sets index 6 to red. `ultimate_palette_reset()` restores the built-in palette; none of these calls writes flash or a palette file. On firmware without the palette commands, each call returns `ULTIMATE_ERR_NOT_SUPPORTED`; test the result before using data from `ultimate_palette_get()`.

USE ULTIMATE 64 TURBO CONTROL

```
uint8_t speed;
uint8_t err;

if (ultimate_turbo_available()) {
    speed = ultimate_turbo_get();
    err = ultimate_turbo_set(U64_SPEED_4MHZ);
    err = ultimate_turbo_badlines(0);
    err = ultimate_turbo_badlines(1);
    err = ultimate_turbo_set(speed);
}
```

OUTPUT

```
AVAILABLE = 1
PREVIOUS SPEED = U64_SPEED_1MHZ
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK
```

`ultimate_turbo_available()` is the capability check and `ultimate_turbo_get()` reads the index. The set call changes only speed. The two badline calls explicitly show both forms: zero disables badlines and non-zero restores them. The final set restores the speed read at entry.

CHANGE AND READ THE CURRENT DIRECTORY

```
char path[128];
uint16_t length;
uint8_t err;

err = ultimate_chdir("/usb1/games");
err = ultimate_getpath(path, sizeof path, &length);
err = ultimate_getpath(path, sizeof path, NULL);
```

OUTPUT

```
path = "/USB1/GAMES"
length = 11
```

The change-directory call selects an absolute or relative firmware path. The paired `getpath` calls show its optional output: receive the stored length through `&length`, or pass `NULL` when the terminated path is enough.

Read every directory entry

```
char name[64];
uint8_t attrib;
uint8_t err = ultimate_opendir();

while (err == ULTIMATE_OK) {
    err = ultimate_readdir(name, sizeof name, &attrib);
    if (err == ULTIMATE_OK) puts(name);
}
if (err == ULTIMATE_END) err = ULTIMATE_OK;
```

OUTPUT

```
DATA
BIG.BIN
README.TXT
```

`ultimate_opendir()` starts one live exchange and each `ultimate_readdir()` releases one entry. `ULTIMATE_END` is normal. Issue no unrelated call inside the loop.

When attributes are not needed, pass the documented optional value:

```
err = ultimate_readdir(name, sizeof name, NULL);
```

OUTPUT

```
name receives the entry; no attribute byte is stored.
```

If a program stops the walk before `ULTIMATE_END`, call `uci_abort()` before issuing another command.

Make a directory, and return to the configured one

```
err = ultimate_mkdir("saves");
err = ultimate_home();
```

OUTPUT

```
SAVES is created inside the current directory.
```

`ultimate_mkdir()` creates one directory in the current one. `ultimate_home()` changes to the directory the machine's owner configured as its home. Firmware that has no configured home returns `ULTIMATE_ERR_NOT_SUPPORTED`.

OPEN, READ AND CLOSE A FILE

```
uint8_t buf[128];
uint16_t got;
uint8_t err;
uint8_t close_err;

err = ultimate_open("readme.txt", DOS_FA_READ);
if (err == ULTIMATE_OK) {
    err = ultimate_seek(0);
    if (err == ULTIMATE_OK)
        err = ultimate_read(buf, sizeof buf, &got);
    close_err = ultimate_close();
    if (err == ULTIMATE_OK) err = close_err;
}
```

OUTPUT

```
got = 128
err = ULTIMATE_OK
```

`ultimate_open()` establishes the firmware's current file. `ultimate_seek()` selects an absolute position, `ultimate_read()` reads from it, and `ultimate_close()` releases the same file. A short read is end-of-file, not an error; `got` is the number stored.

The byte-count output is optional:

```
err = ultimate_read(buf, sizeof buf, NULL);
```

OUTPUT

```
buf receives bytes and the file advances; the count is discarded.
```

Create, write and delete a file

```
static const uint8_t message[] = {0x4f,0x4b,13};

err = ultimate_open("result.txt",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
    err = ultimate_write(message, sizeof message);
    close_err = ultimate_close();
    if (err == ULTIMATE_OK) err = close_err;
}
err = ultimate_delete("result.txt");
```

OUTPUT

RESULT.TXT: WROTE 3 BYTES, CLOSED, DELETED

The open flags create or replace the firmware-side file. `ultimate_write()` appends the specified bytes at its current position; `ultimate_delete()` removes the named file after it is closed.

ASK WHAT A FILE IS

```
ultimate_fileinfo info;
uint16_t year;

err = ultimate_stat("readme.txt", &info);
if (err == ULTIMATE_OK) {
    year = 1980 + (info.date » 9);
    printf("%s: %lu bytes, %u\n",
        info.name, info.size, year);
}
```

OUTPUT

README.TXT: 1024 BYTES, 2026

ultimate_stat() takes a name in the current directory, or a path. ultimate_fstat() answers the same question about the file ultimate_open() left open, and takes no name:

```
err = ultimate_open("readme.txt", DOS_FA_READ);
if (err == ULTIMATE_OK) {
    err = ultimate_fstat(&info);
    ultimate_close();
}
```

OUTPUT

info describes the open file.

The reply has this layout on the wire and is received straight into the structure:

size	length in bytes
date	year less 1980 in bits 15–9, month in 8–5, day in 4–0
time	hour in 15–11, minute in 10–5, two-second units in 4–0
extension	three bytes, space padded, with no terminator
attrib	the DOS_ATTR_* bits, as ultimate_readdir() reports
name	the name, NUL-terminated by the SDK

date and time are the packed forms used by FAT, the File Allocation Table filesystem the Ultimate's storage uses, and they are passed through without conversion. The example above unpacks the year; the other fields come out the same way, with a shift and a mask.

A reply shorter than the fixed header is `ULTIMATE_ERR_PROTOCOL`, because a size read out of half of its four bytes would look like a real answer.

RENAME AND COPY A FILE

```
err = ultimate_rename("old.txt", "new.txt");
err = ultimate_copy("new.txt", "backup.txt");
```

OUTPUT

OLD.TXT becomes NEW.TXT; BACKUP.TXT is a copy of it.

Both names are in the current directory unless they carry a path. The Ultimate performs the copy itself, so no bytes pass through the C64 and a large file costs no more C64 time than a small one.

LOAD AND SAVE C64 MEMORY

```
uint16_t end;

err = ultimate_load("game.prg", 0);
end = ultimate_last_end();

err = ultimate_load("font.prg", 0xC000);
end = ultimate_last_end();

err = ultimate_bload("font.bin", 0xC000, 2048);
err = ultimate_save("screen.bin", 0x0400, 1000);
```

OUTPUT

GAME.PRG END = address after loaded data
FONT.BIN: 2048 bytes loaded at \$C000
SCREEN.BIN: 1000 bytes saved

The first `ultimate_load()` uses the PRG header address; the second shows the explicit-address form. Both consume the header, and `ultimate_last_end()` returns the address following the last byte. `ultimate_bload()` reads bounded raw bytes without a header.

`ultimate_save()` replaces a file with the named C64 memory range.

MOUNT A DISK IMAGE

The Ultimate emulates disk drives, and mounting puts a disk image file into one of them. The drive is named by the number it answers to on the serial bus, 8 or 9, rather than by a slot. That bus is the IEC bus, after the International Electrotechnical Commission, and the number is the one a BASIC program gives in `LOAD"NAME",8`.

```
err = ultimate_mount(8, "games/pitfall.d64");
err = ultimate_swap(8);
err = ultimate_unmount(8);

err = ultimate_mount(ULTIMATE_DRIVE_LAST, "demo.d64");
```

OUTPUT

```
PITFALL.D64 is mounted in the drive answering as device 8.
```

The firmware picks the image type from the extension. `.D64`, `.D71` and `.D81` hold the sectors of a 1541, 1571 and 1581 disk; `.G64` and `.G71` hold the raw track data that a copy-protected disk needs. Any other extension is refused.

`ultimate_swap()` steps to the next image of a multi-image set, the way the Ultimate's own menu does.

`ULTIMATE_DRIVE_LAST` is zero, and asks for whichever drive was mounted into last, or drive A when nothing has been. A program that uses it works without knowing how its user configured the machine, and `ultimate_drive_info()` reports the numbers actually in use.

A drive that is switched off does not exist as far as these calls are concerned; mounting into one returns `ULTIMATE_ERR_DEVICE`. The next section switches drives on.

CONTROL THE MACHINE AND ITS DRIVES

```
ultimate_drives drives;
uint8_t running;
uint8_t i;

err = ultimate_drive_info(&drives);
if (err == ULTIMATE_OK)
    for (i = 0; i < drives.count; ++i)
        printf("type %u device %u power %u\n",
            drives.drive[i].type, drives.drive[i].device,
            drives.drive[i].power);

err = ultimate_drive_enable(ULTIMATE_DRIVE_A, 1);
err = ultimate_drive_power(ULTIMATE_DRIVE_A, &running);
```

OUTPUT

```
TYPE 0 DEVICE 8 POWER 1
TYPE 2 DEVICE 9 POWER 0
running = 1
```

`ultimate_drive_info()` fills in every drive the machine has. `type` is a `CTRL_DRVTYPE_*` code: 0 for a 1541, 1 for a 1571, 2 for a 1581, and `$0F` for SoftwareIEC. `device` is the number to give `ultimate_mount()`. `power` is 1 while the drive is running.

`ultimate_drive_enable()` switches a drive on with a non-zero second argument and off with zero. `ultimate_drive_power()` reports the same state through a pointer, which may not be `NULL`.

Note: Two numbering schemes meet here. `ultimate_drive_enable()` and `ultimate_drive_power()` take `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`, which name a physical drive. `ultimate_mount()` takes the device number the drive answers to on the IEC bus. `ultimate_drive_info()` connects the two: each record carries both.

Reset the machine, or open its menu

```
err = ultimate_freeze();
err = ultimate_reboot();
/* nothing after this line runs */
```

Careful: `ultimate_reboot()` resets the C64. The program that called it stops existing part way through the call, so no line after it runs. `ultimate_freeze()` stops the C64 and shows the Ultimate's menu. It returns when whoever is at the keyboard leaves that menu, which can be minutes.

Read the RAM disk layout

```
uint8_t ramdisk[CTRL_RAMDISK_BYTES];

err = ultimate_ramdisk_info(ramdisk);
```

OUTPUT

```
ramdisk holds four two-byte records.
```

The reply is `CTRL_RAMDISK_DRIVES` records of a drive type code and a size in 64K units, so the buffer must hold `CTRL_RAMDISK_BYTES`. A type of zero is a slot with nothing in it. These are the RAM disks GEOS MegaPatch uses.

READ AND SET THE CLOCK

The Ultimate has a battery-backed clock. It hands the time back as text, because text is what the firmware formats; nothing in the SDK parses it back into numbers.

```
char now[ULTIMATE_TIME_BUFFER];
uint16_t length;

err = ultimate_get_time(ULTIMATE_TIME_PLAIN,
    now, sizeof now, &length);
if (err == ULTIMATE_OK) puts(now);

err = ultimate_get_time(ULTIMATE_TIME_WEEKDAY,
    now, sizeof now, NULL);
if (err == ULTIMATE_OK) puts(now);
```

OUTPUT

```
2026/08/22 14:30:00
SAT 2026/08/22 14:30:00
```

ULTIMATE_TIME_PLAIN asks for the date and time; ULTIMATE_TIME_WEEKDAY puts the weekday in front of it. ULTIMATE_TIME_BUFFER is a buffer size that fits either form. The final parameter is optional in the usual way. A buffer too small for the answer still returns ULTIMATE_OK, with the part that fitted, as `ultimate_getpath()` does.

Set it

```
err = ultimate_set_time(126, 8, 22, 14, 30, 0);
```

OUTPUT

```
The clock is set to 2026/08/22 14:30:00.
```

Setting takes numbers rather than text, in the order year, month, day, hour, minute, second. The year is the year less 1900, so 2026 is 126. That is the firmware's own encoding. The other five fields are the numbers they appear to be.

MOVE BYTES THROUGH THE REU

```
uint16_t banks;

if (ultimate_reu_available()) {
    banks = ultimate_reu_size();
    err = ultimate_reu_stash(0x0400, 0, 1000);
    err = ultimate_reu_fetch(0x0400, 0, 1000);
}
```

OUTPUT

```
banks = installed REU size in 64K banks
1000 bytes stashed and fetched: ULTIMATE_OK
```

Availability is separate from size, which is returned in 64K banks. Stash copies C64 RAM to the REU; fetch copies the same range back. A DMA length of zero means 65536

bytes.

Move the current file directly to and from the REU

File to REU

```
err = ultimate_open("assets.bin", DOS_FA_READ);
if (err == ULTIMATE_OK) {
    err = ultimate_seek(0);
    if (err == ULTIMATE_OK)
        err = ultimate_reu_load(0, 65536UL);
    ultimate_close();
}
```

OUTPUT

ASSETS.BIN TO REU \$000000: 65536 BYTES

REU to file

```
err = ultimate_open("capture.bin",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
    err = ultimate_reu_save(0, 65536UL);
    ultimate_close();
}
```

OUTPUT

REU \$000000 TO CAPTURE.BIN: 65536 BYTES

These functions take no filename or file handle. The first flow opens the firmware's current DOS file, selects its first byte, then copies it to REU address zero. The second flow makes a different current file and copies REU bytes into it. Close the same firmware-side file after either transfer.

INSPECT NETWORK INTERFACES

```
uint8_t count;
uint8_t mac[6];
uint8_t ipconfig[12];

err = ultimate_net_ifcount(&count);
if (err == ULTIMATE_OK && count != 0) {
    err = ultimate_net_macaddr(0, mac);
    err = ultimate_net_ipconfig(0, ipconfig);
}
```

OUTPUT

```
count = number of network interfaces
mac = six bytes; ipconfig = twelve bytes
```

The count call reports available interfaces. The next two calls read the six-byte MAC address and the twelve-byte IPv4 configuration for interface zero into caller-owned buffers. MAC stands for Media Access Control, and the address is the one built into the interface.

Change an interface's address

```
err = ultimate_net_ipconfig(0, ipconfig);
if (err == ULTIMATE_OK) {
    ipconfig[3] = 60;
    err = ultimate_net_setip(0, ipconfig);
}
```

OUTPUT

```
The interface keeps its netmask and gateway and answers on .60 instead.
```

The block `ultimate_net_setip()` writes is the one `ultimate_net_ipconfig()` hands out: four bytes of address, then four of netmask, then four of gateway. Read the block, change the part you want and write it back.

This changes the running configuration only. Nothing is written to the Ultimate's stored settings, and the machine is already on the network by the time a C64 program runs. Neither the firmware nor the SDK checks the address against anything else on the segment.

USE TCP AND UDP SOCKETS

TCP, the Transmission Control Protocol, carries an ordered stream over a connection. UDP, the User Datagram Protocol, sends separate messages with no connection and no ordering. Both are opened the same way here and both give back a handle.

```
uint8_t handle;
uint8_t buf[128];
uint16_t got;
uint16_t sent;
static const uint8_t message[] = {0x48,0x49,13};

err = ultimate_net_connect("192.168.1.50", 6502, &handle);
if (err == ULTIMATE_OK) {
    err = ultimate_net_write(handle, message,
        sizeof message, &sent);
    do {
        err = ultimate_net_read(handle, buf, sizeof buf, &got);
    } while (err == ULTIMATE_OK && got == 0);
    if (err != ULTIMATE_END)
        ultimate_net_close(handle);
}

err = ultimate_net_udp("192.168.1.51", 6502, &handle);
if (err == ULTIMATE_OK)
    err = ultimate_net_close(handle);
```

OUTPUT

```
TCP: 3 BYTES SENT; REPLY BYTES RETURNED IN got
UDP HANDLE OPENED AND CLOSED
```

`ultimate_net_connect()` opens TCP; `ultimate_net_udp()` opens UDP. Both return a handle. Write reports bytes accepted through `sent`; read reports bytes stored through `got`. A successful zero-byte read means “not yet”, so poll. `ULTIMATE_END` means the peer closed and the handle is already released; call `ultimate_net_close()` only for a handle still owned by the program.

FETCH A URL IN ONE CALL

```
static const char url[] = {
    0x68,0x74,0x74,0x70,0x3a,0x2f,0x2f,
    0x31,0x39,0x32,0x2e,0x31,0x36,0x38,0x2e,0x31,
    0x2e,0x35,0x30,0x2f,0
};
uint8_t reply[512];
uint16_t got;

err = ultimate_http_get(url, reply, sizeof reply, &got);
```

OUTPUT

```
err = ULTIMATE_OK
got = HTTP response-body bytes stored in reply
```

`ultimate_http_get()` creates, exchanges and closes one GET request. The explicit bytes prevent cc65 from changing lowercase URL characters to the C64 character set. `got` is the body bytes retained; a larger body returns `ULTIMATE_ERR_TRUNCATED` and has no continuation.

BUILD A MULTI-STEP HTTP REQUEST

Opening the request yourself is what adding a header needs. This example reuses `url`, `reply` and `got` from the preceding one.

```

uint8_t request;
uint8_t close_err;

err = ultimate_http_open(HTTP_VERB_GET, url, &request);
if (err == ULTIMATE_OK) {
    err = ultimate_http_header(request, "accept: text/plain");
    if (err == ULTIMATE_OK)
        err = ultimate_http_exchange(request, HTTP_BODY_NONE,
            reply, sizeof reply, &got);
    close_err = ultimate_http_close(request);
    if (err == ULTIMATE_OK) err = close_err;
}

err = ultimate_http_free_all();

```

OUTPUT

```

GET WITH ONE HEADER: RESPONSE STORED
ALL HTTP OBJECTS FREED

```

`ultimate_http_open()` returns the request handle that header, exchange and close all use. `HTTP_VERB_GET` is one of the `HTTP_VERB_*` codes; `HTTP_VERB_POST`, `HTTP_VERB_PUT` and the rest are in `uci_protocol.h`. The header line is one string in key: value form.

This request sends no body, so the exchange is given `HTTP_BODY_NONE`. The next section builds a body and passes its handle there instead.

Close every handle you open. `ultimate_http_free_all()` recovers slots that an earlier program abandoned.

BUILD A REQUEST BODY

A POST or a PUT needs a body. The body is built inside the Ultimate rather than in C64 memory: create a slot, add one field at a time, then give its handle to `ultimate_http_exchange()`. Only one key and one value are in C64 memory at any moment, which is what lets a 38K machine post an object larger than it could hold.

HTTP_BODY_JSON_OBJECT	a JSON object
HTTP_BODY_JSON_ARRAY	a JSON array
HTTP_BODY_URL_ENCODED	form encoding, as an HTML form sends it
HTTP_BODY_BINARY	raw bytes
HTTP_BODY_NONE	no body; pass this to the exchange

JSON is JavaScript Object Notation, the text format most web services take.

```
/* Explicit bytes, so cc65 does not change their case. */
static const char k_name[] = {0x6e,0x61,0x6d,0x65,0};
static const char k_score[] = {0x73,0x63,0x6f,0x72,0x65,0};
static const char v_ada[] = {0x41,0x44,0x41,0};
uint8_t body;
uint8_t request;

err = ultimate_http_body(HTTP_BODY_JSON_OBJECT, &body);
if (err == ULTIMATE_OK) {
    err = ultimate_http_body_string(body, k_name, v_ada);
    err = ultimate_http_body_int(body, k_score, 4200);

    err = ultimate_http_open(HTTP_VERB_POST, url, &request);
    if (err == ULTIMATE_OK) {
        err = ultimate_http_exchange(request, body,
                                     reply, sizeof reply, &got);
        ultimate_http_close(request);
    }
    ultimate_http_body_free(body);
}
```

OUTPUT

```
POSTED {"name":"ADA","score":4200}; the reply is in reply.
```

Keys and string values go on the wire as given, and cc65 changes the case of a plain C literal exactly as it does for a URL. Build them as bytes, or fold the case back at run time.

Add a boolean, and nest a container

```
err = ultimate_http_body_bool(body, k_alive, 1);

err = ultimate_http_body_object(body, k_stats);
err = ultimate_http_body_int(body, k_score, 4200);
err = ultimate_http_body_up(body);
err = ultimate_http_body_int(body, k_lives, 3);

err = ultimate_http_body_array(body, k_runs);
err = ultimate_http_body_int(body, k_score, 1);
err = ultimate_http_body_up(body);
```

OUTPUT

```
{"alive":true,"stats":{"score":4200},"lives":3,"runs":[1]}
```

Adding an object or an array enters it. Every key added afterwards goes inside, until `ultimate_http_body_up()` steps back out to the parent. Inside an array the key is ignored, so the value alone is what is added. Any non-zero value is true.

Send raw bytes instead

```
static const uint8_t block[] = {1,2,3,4};

err = ultimate_http_body(HTTP_BODY_BINARY, &body);
err = ultimate_http_body_binary(body, block, sizeof block);
err = ultimate_http_body_binary(body, block, sizeof block);

err = ultimate_http_body_clear(body);
err = ultimate_http_body_free(body);
```

OUTPUT

```
The body holds eight bytes, then none; the slot is returned.
```

Successive calls to `ultimate_http_body_binary()` **append**. `ultimate_http_body_clear()` empties a body and keeps the slot and its format; `ultimate_http_body_free()` returns the slot.

A `HTTP_BODY_BINARY` body takes `ultimate_http_body_binary()` and nothing else, and the JSON and form formats take everything else. The firmware answers the wrong combina-

tion with `ULTIMATE_ERR_DEVICE`.

A key is copied through a staging buffer, so it is limited to `ULTIMATE_HTTP_KEY_MAX` bytes, which is 34. A longer key returns `ULTIMATE_ERR_INVALID_ARGUMENT` and never reaches the wire. Values are not limited by that buffer, and a string value is at most 255 bytes, which is the firmware's own length byte.

Careful: The firmware has sixteen slots for the whole machine, and a program that crashes returns none of them. Free every body and every request you create. `ultimate_http_free_all()` takes back headers and bodies together, and is how a program recovers slots an earlier one abandoned.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
uint8_t present = uci_signature_present();
uint8_t ident = uci_ident_register();
uint8_t old_timeout;

err = uci_init();
old_timeout = uci_get_timeout();
uci_set_timeout(UCI_TIMEOUT_FOREVER);
/* run the deliberately unbounded command */
uci_set_timeout(old_timeout);
err = uci_abort();
```

OUTPUT

```
present = 1
ident = command-interface identification byte
abort = ULTIMATE_OK
```

The signature and identification-register calls are immediate register reads. `uci_init()` performs the functional startup check. Timeout zero means wait forever; save and restore the old value. `uci_abort()` releases any exchange still in progress and is safe when the interface is already idle.

EXECUTE A RAW COMMAND WITH AND WITHOUT ARGUMENTS

```
uci_request req = {0};
uint8_t reply[64];
uint8_t status[16];
uint8_t format = 0;
uint8_t sub = CTRL_HWINFO_MODEL;

req.target = UCI_TARGET_DOS1;
req.command = UCI_CMD_IDENTIFY;
req.data = reply;
req.datamax = sizeof reply;
req.status = status;
req.statusmax = sizeof status;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_DOS1;
req.command = DOS_CMD_GET_TIME;
req.args = &format;
req.arglen = 1;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_CONTROL;
req.command = CTRL_CMD_GET_HWINFO;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

req.args = &sub;
req.arglen = 1;
err = uci_exec(&req);
```

OUTPUT

```
IDENTIFY: reply contains target name
GET_TIME: reply contains formatted time
GET_HWINFO: reply contains model name
```

The identify request has no arguments: zero initialization leaves args, payload and their lengths empty. The time request supplies its optional format byte. The hardware-info request is sent first without its optional selector and then with CTRL_HWINFO_MODEL; both forms request the model, which is what an omitted optional argument means on the wire. A payload is a second independent span; set it only when required.

To discard reply data or status deliberately, leave the corresponding pointer and capacity zero:

```
req.data = NULL;
req.datamax = 0;
req.status = NULL;
req.statusmax = 0;
err = uci_exec(&req);
```

OUTPUT

```
req.datalen = 0
req.statuslen = 0
```

READ A RAW MULTI-BLOCK REPLY

```
err = uci_exec_first(&req);
while (err == ULTIMATE_OK) {
    process(req.data, req.datalen);
    if (!uci_more_blocks()) break;
    err = uci_exec_next(&req);
}
```

OUTPUT

```
Each iteration receives one reply block; the loop ends when
uci_more_blocks() = 0
```

The first call submits the request and returns one reply block. The boolean call reports whether another follows; the next call releases the current block and receives another. Do not issue an unrelated command inside this loop.

DECODE AND INSPECT RAW STATUS

```
static const uint8_t raw[] = {
    0x34,0x30,0x34,0x20,0x4e,0x4f,0x54,
    0x20,0x46,0x4f,0x55,0x4e,0x44
};
uint8_t expected;
uint16_t device_code;

expected = uci_status_format(UCI_TARGET_HTTP);
err = uci_decode_status(UCI_TARGET_HTTP, raw, sizeof raw);
device_code = uci_last_device_code();
```

`uci_status_format()` describes the target's expected encoding. `uci_decode_status()` decodes actual bytes and records their numeric status; `uci_last_device_code()` retrieves it. The decoder accepts an empty status as success, so this is also the form without status bytes:

```
err = uci_decode_status(UCI_TARGET_DOS1, NULL, 0);
```

OUTPUT

```
expected = UCI_STATUS_FMT_DECIMAL3
err = ULTIMATE_ERR_DEVICE
device_code = 404
empty DOS status = ULTIMATE_OK
```

CHAPTER 4

PROGRAMMING IN ASSEMBLY

- The register and shared-variable conventions
- Relocating the SDK's RAM and zero page
- Every service entry point by example
- Raw request blocks with optional spans

THE CALLING CONVENTION

Include `ultimate.inc` and call the unprefixed entry points. Unless an entry below says otherwise, a service returns an `ULTIMATE_*` result in A and may change A, X, Y and the flags. A byte argument uses A; a pointer argument uses low byte in A and high byte in X; multi-byte service arguments live in the exported `ult_*` variables.

The examples use these two ordinary ca65 macros to keep pointer and word setup visible without repeating four instructions each time:

```
.include "ultimate.inc"

.macro setptr location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro

.macro setword location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro
```

OUTPUT

```
setptr ult_buf, text stores the address of text in ult_buf.
```

`setptr` and `setword` emit the same bytes; their different names make the purpose clear at each call site. Result branches always compare A immediately, before another instruction replaces it.

Assembly output boxes show the returned register or memory values from a representative successful run. They omit screen-printing routines so the SDK operation itself stays visible.

With the C64 character map, lowercase source letters assemble to the uppercase byte values shared by PETSCII and ASCII. Paths, filenames and HTTP text therefore appear in lowercase source below but reach the firmware in uppercase.

PLACE THE SDK'S WORKING MEMORY

The SDK's zero-page block starts at UCI_ZP, which defaults to \$FB. Its size is UCI_ZP_SIZE. The normal variable block occupies UCI_VARS_SIZE bytes in the BSS segment, which is where the linker puts variables that have no starting value. Define both bases when assembling every SDK source to choose fixed locations instead:

```
ca65 -D UCI_VARS=$CF00 -D UCI_ZP=$A3 uci_core.s
```

OUTPUT

```
UCI variables begin at $CF00; zero-page workspace begins at $A3.
```

With UCI_VARS defined, the core emits no BSS; its variables become equates from that address. The SDK installs no interrupt handler and is not re-entrant, so an interrupt routine must not call it while foreground code is inside a service.

INITIALIZE, DETECT AND DESCRIBE THE MACHINE

```
jsr ultimate_init
cmp #ULTIMATE_OK
bne failed

jsr ultimate_available
beq unavailable

lda #<capabilities
ldx #>capabilities
jsr ultimate_detect
cmp #ULTIMATE_OK
bne failed

capabilities: .res 4
```

OUTPUT

```
A = ULTIMATE_OK
capabilities records the detected targets.
```

ultimate_init performs startup. ultimate_available returns a boolean rather than a result code. ultimate_detect takes the address of the four-byte capability block directly in A/X.

Identify a target with and without the optional length output

```
setptr ult_buf, text
setword ult_buflen, 64
setptr ult_outlen, text_length
lda #UCI_TARGET_DOS1
jsr ultimate_identify

setword ult_outlen, 0
lda #UCI_TARGET_DOS1
jsr ultimate_identify

text: .res 64
text_length: .res 2
```

OUTPUT

```
text = "ULTIMATE-II DOS V1.2"
text_length = 20 with the pointer; no length is stored with zero.
```

Both calls terminate the text in ult_buf. The first stores its length through ult_outlen; zero disables that optional output.

Read the model with and without the optional length output

```
setptr ult_buf, text
setword ult_buflen, 64
setptr ult_outlen, text_length
jsr ultimate_get_model

setword ult_outlen, 0
jsr ultimate_get_model
```

OUTPUT

```
text = "Ultimate 64 Elite"
text_length is optional in the same way as IDENTIFY.
```

ultimate_get_model uses the same buffer contract but needs no target argument. Its underlying CTRL_CMD_GET_HWINFO command is marked deprecated by the firmware documentation; this existing wrapper remains for compatibility.

Read SID addresses through deprecated HWINFO

```
lda #<sid_info
ldx #>sid_info
jsr ultimate_legacy_get_sid_info
cmp #ULTIMATE_OK
bne failed

lda sid_info + ULTIMATE_SID_INFO_COUNT
; iterate this many ULTIMATE_SID_RECORD_SIZE-byte records
ldx #ULTIMATE_SID_INFO_RECORDS
lda sid_info + ULTIMATE_SID_PRIMARY_ADDRESS,x
sta first_sid_address
lda sid_info + ULTIMATE_SID_PRIMARY_ADDRESS + 1,x
sta first_sid_address + 1

sid_info: .res ULTIMATE_SID_INFO_SIZE
first_sid_address: .res 2
```

OUTPUT

```
sid_info count = 4
primary addresses = $D400, $D400, $D400, $D400
```

Records begin at `ULTIMATE_SID_INFO_RECORDS`. Within each record, `ULTIMATE_SID_PRIMARY_ADDRESS` and `ULTIMATE_SID_SECONDARY_ADDRESS` name two little-endian words, and `ULTIMATE_SID_TYPE` names the raw type byte. The type is not a 6581 or 8580 identifier. Firmware still implements this query, but marks HWINFO deprecated and publishes no replacement C64-side UCI command. The legacy name is deliberate; handle `ULTIMATE_ERR_NOT_SUPPORTED` and read no record unless the call returned `ULTIMATE_OK`.

Turn a result into text

```
lda error_code
jsr ultimate_strerror
sta message_ptr
stx message_ptr + 1

error_code: .res 1
message_ptr: .res 2
```

OUTPUT

message_ptr points to the PETSCII text for error_code.

ultimate_strerror returns the PETSCII message pointer in A/X; it does not return a result code.

READ AND CHANGE THE PALETTE

```
lda #<palette
ldx #>palette
jsr ultimate_palette_get

lda #<palette
ldx #>palette
jsr ultimate_palette_set

lda #6
sta ult_color
lda #255
sta ult_color + 1
lda #0
sta ult_color + 2
sta ult_color + 3
jsr ultimate_palette_set_color

jsr ultimate_palette_reset

palette: .res UCI_PALETTE_BYTES
```

OUTPUT

GET fills 48 bytes; SET COLOUR 6 TO RED and RESET return ULTIMATE_OK.

Get and set take a direct pointer in A/X. The single-colour entry reads index, red, green and blue from the four consecutive ult_color bytes. Reset restores the built-in live palette. Firmware without these commands returns ULTIMATE_ERR_NOT_SUPPORTED; branch on A before using the 48-byte result from ultimate_palette_get.

USE TURBO AND BADLINE CONTROL

```
jsr ultimate_turbo_available
beq no_turbo

jsr ultimate_turbo_get
sta old_speed

lda #U64_SPEED_4MHZ
jsr ultimate_turbo_set

lda #0
jsr ultimate_turbo_badlines
lda #1
jsr ultimate_turbo_badlines

lda old_speed
jsr ultimate_turbo_set

old_speed: .res 1
```

OUTPUT

```
old_speed = previous speed index
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK
```

The available and get calls return values directly. Set returns a result. The two badline calls show both forms explicitly: zero disables them; non-zero restores them. The final set restores the speed read at entry.

CHANGE AND READ THE CURRENT DIRECTORY

```
setptr ult_buf, path
jsr ultimate_chdir

setptr ult_buf, path_buffer
setword ult_buflen, 128
setptr ult_outlen, path_length
jsr ultimate_getpath

setword ult_outlen, 0
jsr ultimate_getpath

path: .byte "/usb1/games", 0
path_buffer: .res 128
path_length: .res 2
```

OUTPUT

```
path_buffer = "/USB1/GAMES"
path_length = 11 when the optional pointer is enabled.
```

ultimate_chdir consumes the terminated path through ult_buf. The two getpath calls show the optional length pointer enabled and disabled.

Read every directory entry

```
    jsr ultimate_opendir
    cmp #ULTIMATE_OK
    bne failed

next_entry:
    setptr ult_buf, name
    setword ult_buflen, 64
    jsr ultimate_readdir
    cmp #ULTIMATE_END
    beq directory_done
    cmp #ULTIMATE_OK
    bne failed
    lda ult_attr
    ; use name and its DOS_ATTR_* bits here
    jmp next_entry

name: .res 64
```

OUTPUT

name receives one entry per call; A becomes ULTIMATE_END after the last.

Each readdir releases one reply block and writes attributes to ult_attr. Issue no other command inside the walk. Call uci_abort if leaving before ULTIMATE_END.

Make a directory, and return to the configured one

```
    setptr ult_buf, dir_name
    jsr ultimate_mkdir

    jsr ultimate_home

dir_name: .byte "saves", 0
```

OUTPUT

SAVES is created; A = ULTIMATE_OK from each call.

ultimate_mkdir takes the terminated name through ult_buf and measures it itself.

ultimate_home takes nothing at all.

OPEN, SEEK, READ, WRITE AND CLOSE FILES

```
    setptr ult_buf, read_name
    lda #DOS_FA_READ
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed

    lda #0
    sta ult_num
    sta ult_num + 1
    sta ult_num + 2
    sta ult_num + 3
    jsr ultimate_seek

    setptr ult_buf, file_buffer
    setword ult_buflen, 128
    setptr ult_outlen, bytes_read
    jsr ultimate_read
    jsr ultimate_close

read_name: .byte "readme.txt", 0
file_buffer: .res 128
bytes_read: .res 2
```

OUTPUT

file_buffer receives up to 128 bytes; bytes_read receives the actual count.

Open establishes the firmware's current file. Seek selects its absolute position. Read stores bytes and, when ult_outlen is non-zero, their count. Close releases that same firmware-side file.

The count output is optional:

```
; With a readable file already open:
setptr ult_buf, file_buffer
setword ult_buflen, 128
setword ult_outlen, 0
jsr ultimate_read
```

OUTPUT

file_buffer receives bytes; no byte count is stored.

Create, write and delete a file

```
setptr ult_buf, write_name
lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
jsr ultimate_open
cmp #ULTIMATE_OK
bne failed

setptr ult_buf, message
setword ult_buflen, message_end - message
jsr ultimate_write
jsr ultimate_close

setptr ult_buf, write_name
jsr ultimate_delete

write_name: .byte "result.txt", 0
message: .byte "ok", 13
message_end:
```

OUTPUT

RESULT.TXT: 3 bytes written, closed and deleted; A = ULTIMATE_OK.

The open mask creates or replaces the file. Write consumes ult_buflen bytes from ult_buf; delete consumes the terminated name after the file is closed.

ASK WHAT A FILE IS

```
setptr ult_buf, stat_name
setptr ult_arg2, file_info
jsr ultimate_stat
cmp #ULTIMATE_OK
bne failed

lda file_info + DOS_INFO_SIZE
sta file_size
lda file_info + DOS_INFO_SIZE + 1
sta file_size + 1
lda file_info + DOS_INFO_ATTRIB
and #DOS_ATTR_DIR
bne is_a_directory

stat_name: .byte "readme.txt", 0
file_info: .res ULTIMATE_FILEINFO_SIZE
file_size: .res 2
```

OUTPUT

file_info holds the reply; the name inside it is NUL-terminated.

ult_buf is the name and ult_arg2 is the block the reply is received into. That block must be ULTIMATE_FILEINFO_SIZE bytes. ult_bufen is not an input here: ultimate_stat measures the name itself and overwrites ult_bufen doing it.

ultimate_fstat asks the same question about the file ultimate_open left open, so it takes only the block:

```
setptr ult_arg2, file_info
jsr ultimate_fstat
```

OUTPUT

file_info describes the open file; ult_buf is not read.

The DOS_INFO_* equates in uci_protocol.inc name the fields inside the block:

DOS_INFO_SIZE	+\$00, size in bytes, 32-bit, low byte first
DOS_INFO_DATE	+\$04, year less 1980 in bits 15–9, month 8–5, day 4–0
DOS_INFO_TIME	+\$06, hour 15–11, minute 10–5, two-second units 4–0
DOS_INFO_EXTENSION	+\$08, three bytes, space padded
DOS_INFO_ATTRIB	+\$0B, the DOS_ATTR_* bits
DOS_INFO_NAME	+\$0C, the name, terminated by the SDK

Date and time are the packed forms used by FAT, the File Allocation Table filesystem the Ultimate's storage uses, and they arrive unconverted.

RENAME AND COPY A FILE

```

    setptr ult_buf, old_name
    setptr ult_arg2, new_name
    jsr ultimate_rename

    setptr ult_buf, new_name
    setptr ult_arg2, copy_name
    jsr ultimate_copy

old_name: .byte "old.txt", 0
new_name: .byte "new.txt", 0
copy_name: .byte "backup.txt", 0

```

OUTPUT

OLD.TXT becomes NEW.TXT; BACKUP.TXT is a copy of it.

Both take the first name in `ult_buf` and the second in `ult_arg2`. The request block carries two spans and these commands put three runs of bytes on the wire, so the first name's own terminator separates it from the second. The Ultimate performs the copy, so no bytes pass through the C64.

LOAD AND SAVE C64 MEMORY

```
setptr ult_buf, prg_name
setword ult_addr, 0
jsr ultimate_load
jsr ultimate_last_end
sta load_end
stx load_end + 1

setword ult_addr, $C000
jsr ultimate_load

setptr ult_buf, raw_name
setword ult_addr, $C000
setword ult_max, 2048
jsr ultimate_bload

setptr ult_buf, save_name
setword ult_addr, $0400
setword ult_max, 1000
jsr ultimate_save

prg_name: .byte "game.prg", 0
raw_name: .byte "font.bin", 0
save_name: .byte "screen.bin", 0
load_end: .res 2
```

OUTPUT

```
load_end = address after loaded PRG data
FONT.BIN loads at $C000; SCREEN.BIN receives 1000 bytes.
```

An `ult_addr` of zero makes load use the PRG header; `$C000` shows the explicit-address form. `ultimate_last_end` returns the previous load end in A/X. `Bload` consumes no header and obeys `ult_max`; `save` uses `ult_max` as its length.

MOUNT A DISK IMAGE

The drive is named by the number it answers to on the IEC serial bus, 8 or 9, rather than by a slot. `ULTIMATE_DRIVE_LAST` is zero and asks for whichever drive was mounted into last, or drive A when nothing has been.

```

    setptr ult_buf, image_name
    lda #8
    jsr ultimate_mount
    cmp #ULTIMATE_OK
    bne failed

    lda #8
    jsr ultimate_swap

    lda #8
    jsr ultimate_unmount

image_name: .byte "games/pitfall.d64", 0

```

OUTPUT

PITFALL.D64 is mounted, swapped and unmounted; A = ULTIMATE_OK each time.

The device number goes in A and is reloaded before each call, because every entry point returns its result there. `ultimate_mount` takes the image name through `ult_buf` and measures it itself.

The firmware picks the image type from the extension: `.D64`, `.D71` and `.D81` hold the sectors of a 1541, 1571 and 1581 disk, and `.G64` and `.G71` hold raw track data. Mounting into a drive that is switched off returns `ULTIMATE_ERR_DEVICE`.

CONTROL THE MACHINE AND ITS DRIVES

```
lda #<drives
ldx #>drives
jsr ultimate_drive_info
cmp #ULTIMATE_OK
bne failed
lda drives + CTRL_DRVINFO_COUNT
beq no_drives
lda drives + CTRL_DRVINFO_FIRST + 1
sta first_device

lda #ULTIMATE_DRIVE_A
ldx #1
jsr ultimate_drive_enable

lda #ULTIMATE_DRIVE_A
jsr ultimate_drive_power
lda ult_stage
beq drive_is_off

drives: .res CTRL_DRVINFO_BYTES
first_device: .res 1
```

OUTPUT

drives holds a count and one record per drive; ult_stage = 1 when it runs.

ultimate_drive_info takes the block pointer in A/X. The block is a count at CTRL_DRVINFO_COUNT, then CTRL_DRVINFO_RECORD bytes per record from CTRL_DRVINFO_FIRST: type, then the IEC device number, then 1 while the drive is running. On any failure the count is set to zero, so a caller that skips the result check reads no records.

The block must be CTRL_DRVINFO_BYTES long, because the records are not only drives A and B. The firmware reports each occupied IEC bus slot in the same reply, and there are four of those, so the count reaches six on a machine running SoftwareIEC and an IEC printer. A slot has a type of CTRL_DRVTYPE_SOFTIEC or CTRL_DRVTYPE_PRINTER; read the type rather than assuming every record is a disk drive.

ultimate_drive_enable takes the drive in A and the state in X: zero switches it off, and any non-zero value switches it on. ultimate_drive_power takes the drive in A and reports the state in ult_stage, which is 1 while the drive runs.

Note: Two numbering schemes meet here. `ultimate_drive_enable` and `ultimate_drive_power` take `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`, which name a physical drive. `ultimate_mount` takes the device number the drive answers to on the IEC bus. `ultimate_drive_info` connects the two: each record carries both.

Those two entry points accept no drive beyond A and B, because the firmware has one command per drive and it has commands for those two only. That limit is `CTRL_DRIVE_SLOTS` and it is not `CTRL_DRVINFO_MAX`, which is how many records the information reply can carry.

Careful: `ultimate_drive_power` writes the flag to `ult_stage` and uses `ult_stage + 1` to `ult_stage + 3` as scratch. Read the flag before the next SDK call.

Reset the machine, or open its menu

```
jsr ultimate_freeze

jsr ultimate_reboot
; nothing here runs
```

OUTPUT

freeze returns when the menu is left; reboot does not return.

`ultimate_reboot` resets the C64, so the program calling it stops existing part way through the call. `ultimate_freeze` stops the C64 and shows the Ultimate's menu, and returns when whoever is at the keyboard leaves it.

Read the RAM disk layout

```
lda #<ramdisk
ldx #>ramdisk
jsr ultimate_ramdisk_info

ramdisk: .res CTRL_RAMDISK_BYTES
```

OUTPUT

```
ramdisk holds CTRL_RAMDISK_DRIVES two-byte records.
```

Each record is a drive type code and a size in 64K units. A type of zero is a slot with nothing in it. These are the RAM disks GEOS MegaPatch uses.

READ AND SET THE CLOCK

```
setptr ult_buf, time_text
setword ult_buflen, ULTIMATE_TIME_BUFFER
setword ult_outlen, 0
lda #ULTIMATE_TIME_PLAIN
jsr ultimate_get_time

time_text: .res ULTIMATE_TIME_BUFFER
```

OUTPUT

```
time_text = "2026/08/22 14:30:00"
```

A is the format. ULTIMATE_TIME_PLAIN asks for the date and time and ULTIMATE_TIME_WEEKDAY puts the weekday in front of it. ULTIMATE_TIME_BUFFER is a buffer size that fits either form. This is one of the few entry points that reads ult_buflen as an input rather than computing it. A buffer too small still returns ULTIMATE_OK, with the part that fitted.

Set it

```
lda #126    ; 2026, as the year less 1900
sta ult_stage + 0
lda #8      ; month
sta ult_stage + 1
lda #22     ; day
sta ult_stage + 2
lda #14     ; hour
sta ult_stage + 3
lda #30     ; minute
sta ult_stage + 4
lda #0      ; second
sta ult_stage + 5
jsr ultimate_set_time
```

OUTPUT

The clock is set to 2026/08/22 14:30:00; A = ULTIMATE_OK.

The six bytes go in ult_stage in that order. ult_stagelen is not a caller input here; the command always sends six bytes.

Careful: ult_stage is shared. ultimate_mount, ultimate_unmount, ultimate_swap, ultimate_get_time, ultimate_drive_info and ultimate_http_body all overwrite ult_stage + 0, and ultimate_drive_power and the keyed body calls overwrite more of it. Fill it immediately before ultimate_set_time.

MOVE BYTES THROUGH THE REU

```
jsr ultimate_reu_available
beq no_reu
jsr ultimate_reu_size
sta reu_banks
stx reu_banks + 1

setword ult_addr, $0400
lda #0
sta ult_reu
sta ult_reu + 1
sta ult_reu + 2
sta ult_reu + 3
setword ult_reulen, 1000
lda #0
sta ult_reulen + 2
sta ult_reulen + 3
jsr ultimate_reu_stash
jsr ultimate_reu_fetch

reu_banks: .res 2
```

OUTPUT

```
reu_banks = installed REU size in 64K banks
1000 bytes stashed and fetched; A = ULTIMATE_OK.
```

Size returns 64K banks in A/X. Stash copies C64 RAM to the REU; fetch copies it back. Both read the C64 address, 32-bit REU address and 32-bit length from the shared variables.

Move the current file directly to and from the REU

Both transfers below use REU address zero and a 65536-byte length:


```

lda #0
sta ult_reu
sta ult_reu + 1
sta ult_reu + 2
sta ult_reu + 3
sta ult_reulen
sta ult_reulen + 1
lda #1
sta ult_reulen + 2
lda #0
sta ult_reulen + 3

```

OUTPUT

```
ult_reu = $00000000; ult_reulen = 65536.
```

File to REU

```

; Open ASSETS.BIN first; it becomes the current DOS file.
setptr ult_buf, asset_name
lda #DOS_FA_READ
jsr ultimate_open
cmp #ULTIMATE_OK
bne failed
lda #0
sta ult_num
sta ult_num + 1
sta ult_num + 2
sta ult_num + 3
jsr ultimate_seek
jsr ultimate_reu_load
jsr ultimate_close

```

```
asset_name: .byte "assets.bin", 0
```

OUTPUT

```
ASSETS.BIN to REU $000000: 65536 bytes
```

REU to file

```
; Open a writable current file before the transfer.
setptr ult_buf, capture_name
lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
jsr ultimate_open
cmp #ULTIMATE_OK
bne failed
jsr ultimate_reu_save
jsr ultimate_close

capture_name: .byte "capture.bin", 0
```

OUTPUT

```
REU $000000 to CAPTURE.BIN: 65536 bytes
```

The REU address is zero and the length bytes encode 65536. Load and save take no filename: each operates on the firmware's current open file established by the preceding `ultimate_open`.

INSPECT NETWORK INTERFACES

```
jsr ultimate_net_ifcount
cmp #ULTIMATE_OK
bne failed
lda ult_iface
beq no_interfaces

lda #0
sta ult_iface
setptr ult_buf, mac
jsr ultimate_net_macaddr
setptr ult_buf, ipconfig
jsr ultimate_net_ipconfig

mac: .res 6
ipconfig: .res 12
```

OUTPUT

```
ult_iface = interface count; mac receives 6 bytes; ipconfig receives 12.
```

The count is returned in `ult_iface`; the address calls take an interface index there and place fixed-size replies at `ult_buf`.

Change an interface's address

```
lda #0
sta ult_iface
setptr ult_buf, ipconfig
jsr ultimate_net_ipconfig
cmp #ULTIMATE_OK
bne failed

lda #60
sta ipconfig + 3
jsr ultimate_net_setip
```

OUTPUT

The interface keeps its netmask and gateway and answers on .60 instead.

ultimate_net_setip sends back the same twelve bytes ultimate_net_ipconfig returned: four of address, then four of netmask, then four of gateway. It reads ult_iface and ult_buf, and always sends twelve bytes, so ult_buflen is not an input.

This changes the running configuration only; nothing is written to the Ultimate's stored settings.

USE TCP AND UDP SOCKETS

```
setptr ult_buf, host
setword ult_port, 6502
jsr ultimate_net_connect
cmp #ULTIMATE_OK
bne failed

setptr ult_buf, message
setword ult_buflen, message_end - message
jsr ultimate_net_write
; ult_socklen is now the number accepted.
```

```
host: .byte "192.168.1.50", 0
message: .byte "hi", 13
message_end:
```

OUTPUT

TCP handle is in ult_sock; ult_socklen = 3 bytes accepted.

Read and close the TCP socket

```
poll:
    setptr ult_buf, socket_buffer
    setword ult_socklen, 128
    jsr ultimate_net_read
    cmp #ULTIMATE_END
    beq peer_closed
    cmp #ULTIMATE_OK
    bne close_socket
    lda ult_socklen
    ora ult_socklen + 1
    beq poll

close_socket:
    jsr ultimate_net_close

socket_buffer: .res 128
```

OUTPUT

Reply bytes are in socket_buffer; close returns ULTIMATE_OK.

UDP

```
    setptr ult_buf, host
    setword ult_port, 6502
    jsr ultimate_net_udp
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_net_close
```

OUTPUT

UDP handle opens and closes; A = ULTIMATE_OK.

Connect and UDP return the handle in ult_sock. Write takes its length from ult_buflen and reports accepted bytes in ult_socklen. Read takes capacity and returns stored bytes through ult_socklen. A zero-byte success means poll again. ULTIMATE_END means the handle is already gone; do not close that ended handle.

USE THE HTTP CLIENT

```
setptr ult_url, url
setptr ult_buf, http_reply
setword ult_buflen, 512
jsr ultimate_http_get
; ult_httplen is the number stored.
```

```
url: .byte "http://192.168.1.50/", 0
accept_header: .byte "accept: text/plain", 0
http_reply: .res 512
```

OUTPUT

One-call GET stores response bytes; A = ULTIMATE_OK.

Multi-step GET without a body

```
setptr ult_url, url
lda #HTTP_VERB_GET
jsr ultimate_http_open
cmp #ULTIMATE_OK
bne failed

setptr ult_url, accept_header
jsr ultimate_http_header
lda #HTTP_BODY_NONE
sta ult_body
setptr ult_buf, http_reply
setword ult_buflen, 512
jsr ultimate_http_exchange
jsr ultimate_http_close
```

OUTPUT

GET without a body stores response bytes; close returns ULTIMATE_OK.

The one-call GET creates and releases its own slot. The multi-step form stores the request handle in `ult_http`, and header, exchange and close all read it there. `ult_body` holds the body handle, and `HTTP_BODY_NONE` in it means no body. `ultimate_http_free_all` recovers slots abandoned by an earlier program.

The next section creates a body and posts it.

BUILD A REQUEST BODY

The body is built inside the Ultimate: create a slot, add to it one call at a time, then run the exchange. The handle lives in `ult_body`, where `ultimate_http_exchange` already looks for it, so nothing has to carry it around.

```
lda #HTTP_BODY_JSON_OBJECT
jsr ultimate_http_body
cmp #ULTIMATE_OK
bne failed

setptr ult_url, key_name
setptr ult_buf, value_ada
jsr ultimate_http_body_string

setptr ult_url, key_score
setword ult_val, 4200
lda #0
sta ult_val + 2
sta ult_val + 3
jsr ultimate_http_body_int

key_name: .byte "name", 0
key_score: .byte "score", 0
value_ada: .byte "ada", 0
```

OUTPUT

The body holds {"NAME":"ADA","SCORE":4200}.

`ult_url` is the key for every keyed call, and it is the same variable `ultimate_http_open` uses for a URL. `ult_buf` is a string value. `ult_val` is four bytes, low byte first, and all four are sent, because the firmware sign-extends from the last byte it receives.

`ultimate_http_body_bool` reads `ult_val + 0` only, and any non-zero value is true.

Send it

```
setptr ult_url, url
lda #HTTP_VERB_POST
jsr ultimate_http_open
cmp #ULTIMATE_OK
bne failed

setptr ult_buf, http_reply
setword ult_buflen, 512
jsr ultimate_http_exchange
jsr ultimate_http_close
jsr ultimate_http_body_free
```

OUTPUT

The response is in http_reply; ult_httplen is the number stored.

Set ult_buf back to the reply buffer before the exchange. The keyed body calls used it for the last string value, and the exchange uses it for the reply.

Nest an object or an array

```
setptr ult_url, key_stats
jsr ultimate_http_body_object
; keys added here go inside "stats"
jsr ultimate_http_body_up

setptr ult_url, key_runs
jsr ultimate_http_body_array
; inside an array the key is ignored
jsr ultimate_http_body_up
```

OUTPUT

```
{"STATS":{...},"RUNS":[...]}
```

Adding an object or an array enters it. Every key added afterwards goes inside, until ultimate_http_body_up steps back out to the parent.

Send raw bytes instead

```
lda #HTTP_BODY_BINARY
jsr ultimate_http_body

setptr ult_buf, block
setword ult_buflen, block_end - block
jsr ultimate_http_body_binary

jsr ultimate_http_body_clear
jsr ultimate_http_body_free

block: .byte 1, 2, 3, 4
block_end:
```

OUTPUT

The body holds four bytes, then none; the slot is returned.

`ultimate_http_body_binary` is the other entry that reads `ult_buflen` as an input. Successive calls append. `ultimate_http_body_clear` empties a body and keeps its slot and format; `ultimate_http_body_free` returns the slot.

A `HTTP_BODY_BINARY` body takes `ultimate_http_body_binary` and nothing else; the JSON and form formats take everything else. A key is staged through `ult_stage`, so it is limited to `ULTIMATE_HTTP_KEY_MAX` bytes, which is 34. A longer key returns `ULTIMATE_ERR_INVALID_ARGUMENT` and never reaches the wire.

Careful: The firmware has sixteen slots for the whole machine, and a program that crashes returns none of them. Free every body and every request. `ultimate_http_free_all` takes back headers and bodies together.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
jsr uci_present
beq no_interface
jsr uci_ident
sta raw_ident

jsr uci_get_timeout_a
pha
lda #UCI_TIMEOUT_FOREVER
jsr uci_set_timeout_a
jsr run_slow_command
tax
pla
jsr uci_set_timeout_a
txa

jsr uci_last_code
sta device_code
stx device_code + 1
jsr uci_abort
```

OUTPUT

uci_present returns nonzero; raw_ident receives the interface ID.
device_code receives the last target status; abort returns ULTIMATE_OK.

Present and ident are direct register reads. Timeout zero waits forever; this pattern restores the previous budget while preserving the command result. Last code returns the raw target status in A/X. Abort releases an unfinished exchange and is safe while idle.

RAW COMMANDS: OPTIONAL ARGUMENTS

```
jsr uci_req_clear
lda #UCI_TARGET_DOS1
sta uci_req + UCI_REQ_TARGET
lda #UCI_CMD_IDENTIFY
sta uci_req + UCI_REQ_COMMAND
setptr uci_req + UCI_REQ_DATA, raw_reply
setword uci_req + UCI_REQ_DATAMAX, 64
jsr uci_exec_block
jsr uci_req_clear
lda #UCI_TARGET_DOS1
sta uci_req + UCI_REQ_TARGET
lda #DOS_CMD_GET_TIME
sta uci_req + UCI_REQ_COMMAND
setptr uci_req + UCI_REQ_ARGS, time_format
setword uci_req + UCI_REQ_ARGLEN, 1
jsr uci_exec_block
jsr uci_req_clear
lda #UCI_TARGET_CONTROL
sta uci_req + UCI_REQ_TARGET
lda #CTRL_CMD_GET_HWINFO
sta uci_req + UCI_REQ_COMMAND
setptr uci_req + UCI_REQ_DATA, raw_reply
setword uci_req + UCI_REQ_DATAMAX, 64
jsr uci_exec_block
setptr uci_req + UCI_REQ_ARGS, hwinfo_model
setword uci_req + UCI_REQ_ARGLEN, 1
jsr uci_exec_block
time_format: .byte 0
hwinfo_model: .byte CTRL_HWINFO_MODEL
raw_reply: .res 64
```

OUTPUT

IDENTIFY returns the target name.
GET_TIME returns the formatted time.
Both GET_HWINFO forms return the model name.

Clearing the request leaves every optional span empty. The time command then supplies its optional format byte. GET_HWINFO is sent without and then with its model selector, which is what an omitted optional argument means on the wire. Set other spans only when needed.

Use a caller-owned request block

```
    lda #<my_request
    ldx #>my_request
    jsr uci_exec

my_request: .res UCI_REQ_SIZE
```

OUTPUT

```
A = request result; lengths and retained bytes are written into
my_request.
```

`uci_exec` takes the request pointer directly. Use it instead of the global block when the program owns more than one request.

READ A RAW MULTI-BLOCK REPLY

```
lda #<my_request
ldx #>my_request
jsr uci_exec_first
cmp #ULTIMATE_OK
bne failed

next_block:
; use this block's UCI_REQ_DATALEN bytes
lda uci_more
beq all_blocks
jsr uci_exec_next
cmp #ULTIMATE_OK
beq next_block
```

OUTPUT

Each iteration receives one block; uci_more = 0 after the final block.

First submits the request and returns one block. uci_more is the boolean state byte that says whether another follows; next releases the current block and receives it. Issue no unrelated command until the final block or call uci_abort when stopping early.

CHAPTER 5

OTHER TOOLCHAINS

- Building and identifying the blob
- The stable jump table
- The parameter block
- Calling it from BASIC
- Choosing a different base address

BUILD AND IDENTIFY THE BLOB

The blob is the SDK linked as one binary with a table of jump instructions at a fixed address. Any assembler or compiler can call it, because calling it needs no linker and no object format: you load the binary and `jsr` into the table.

`make blob` produces `bindings/blob/build/ultimate-7000.bin`. Load it at `$7000`. The first three bytes are the PETSCII signature UCI; the fourth is the version of the ABI, the calling interface.

Note: The blob has to sit below `$A000`, because `$A000–$BFFF` is BASIC ROM. A program can reach every byte of a blob at `$7000` with the machine exactly as it found it. One that ran past `$A000` would have to switch BASIC ROM out and back around every call.

Careful: `$7000` is inside BASIC's memory, not above it. BASIC owns `$0801–$9FFF`: the program area grows up from `$0801` and strings and arrays grow down from `$9FFF`. A program that has replaced BASIC owns that memory instead and needs to do nothing. A program that leaves BASIC running must lower the top of BASIC memory to the base address before it loads the blob, or BASIC will write over it. *Call The Blob From BASIC*, later in this chapter, shows the two `POKEs` and the `CLR` that do it.

This example checks the default build before calling it:

```
lda $7000
cmp #$D5      ; PETSCII 'U'
bne no_sdk
lda $7001
cmp #$C3      ; PETSCII 'C'
bne no_sdk
lda $7002
cmp #$C9      ; PETSCII 'I'
bne no_sdk
lda $7003
cmp #1        ; ABI version
bne wrong_version
```

OUTPUT

All four comparisons match; execution continues with ABI version 1.

The version byte changes only when an existing entry changes meaning. New entries

are appended, so a program written against version 1 keeps working when the table grows.

CALL THE STABLE JUMP TABLE

Entries begin at base plus \$04. Each entry is a three-byte `jmp`; new entries are appended so published offsets do not move.

```
jsr $7004    ; +$04 uci_init
jsr $701C    ; +$1C ultimate_init
```

OUTPUT

```
A = ULTIMATE_OK after each initialization call.
```

These are the two initialization entries. Use `ultimate_init` for application code; the lower-level `uci_init` entry is exposed for transport users. Both return a result in A.

The two kinds of entry

The table has two halves, and the difference decides how you read a result.

Entries from `+$04` to `+$4C` pass their arguments in registers, the way the assembly chapter's entry points do, and return the result in A and nowhere else.

Entries from `+$4F` upwards take their arguments from the parameter block described in the next section, because a caller cannot pass a filename in A and X. Each of these also stores its result in `bp_result` before returning it in A.

That second group is what a BASIC program can drive, because `SYS` can neither pass a register nor read one back.

The complete offset table is in `bindings/blob/README.md`. Read the offsets there rather than counting entries: the table gained the file information, disk image, clock, machine and HTTP body services after the offsets printed in this chapter were published.

USE THE PARAMETER BLOCK

Service entries that need several arguments use a 512-byte block at base plus \$100. These are the first four fields:

Offset	Field	Purpose
+\$00	bp_result	last service result
+\$05	bp_len	length in or out
+\$29	bp_name	40-byte area for a terminated name
+\$51	bp_reply	256-byte reply area

Later fields extend beyond the first page while retaining their published offsets. Consult the blob README for the complete layout rather than deriving an address from this subset.

CALL THE BLOB FROM BASIC

The page-aligned block gives BASIC a calling convention it can express with POKE, PEEK and SYS. Before any of that, BASIC has to be told to stay out of the memory the blob occupies. Type this first, then load the blob:

```
POKE 56,112 : POKE 55,0 : CLR
```

112 is \$70, the high byte of the base address, and 55 and 56 are the low and high bytes of BASIC's top-of-memory pointer at \$37-\$38. CLR is what makes it take effect, because it resets the string pointer from the new top. BASIC then keeps \$0801-\$6FFF, and the blob's code at \$7000 and its variables at \$9F00 are both out of its reach. Loading the blob without this leaves it where BASIC's strings and arrays will eventually be written.

This program then checks the signature, brings the SDK up, confirms that an Ultimate answered, and prints the size of the RAM expansion:

```
10 B = 28672 : P = B + 256
20 IF PEEK(B)<>213 OR PEEK(B+1)<>195 THEN PRINT "NO SDK":END
25 IF PEEK(B+2)<>201 THEN PRINT "NO SDK":END
30 SYS B + 28
40 SYS B + 82 : IF PEEK(P) THEN PRINT "NO ULTIMATE":END
50 SYS B + 175
60 PRINT "REU BANKS: "; PEEK(P+5) + PEEK(P+6)*256
```

Line 10 sets B to the blob's base, decimal 28672 for \$7000, and P to the parameter block a page above it. Lines 20 and 25 compare the three signature bytes; 213, 195 and 201 are \$D5, \$C3 and \$C9.

Line 30 calls `ultimate_init` at offset \$1C. That is a register entry, so it returns its result in A and writes nothing to the block, and SYS cannot read A. Line 40 therefore calls `getpath` at offset \$52, which is a parameter-block entry: it asks the Ultimate for the current directory and leaves the result in `bp_result`, so `PEEK(P)` is the first result this program can test. A non-zero value there means no Ultimate answered.

Line 50 calls `reu_size` at offset \$AF, which puts the 16-bit bank count in `bp_1en`. Line 60 reads it back as two bytes, low byte first, and prints it. A machine with a 2MB expansion prints 32; one with no expansion prints 0.

Careful: Do not check `PEEK(P)` after a register entry such as \$1C. Those entries never write `bp_result`, so the byte still holds whatever the previous call left there, and in a freshly loaded blob that is zero. The test would pass whether the call worked or not.

CHOOSE A DIFFERENT BASE ADDRESS

Prefer building at the final base: `make -C bindings/blob BASE=6000` links a \$6000 image directly, and VARS and ZP move the SDK's RAM the same way.

When the address is known only at run time, the build also emits a relocation table beside the binary for `reloc.s` to apply. The relocated emulator suite copies the default blob to \$6000, relocates the copy, calls it, and then compares it byte for byte against an image the linker built for \$6000 directly.

CHAPTER 6

APPENDIX

- Result codes

RESULT CODES

0	ULTIMATE_OK	success
1	ULTIMATE_ERR_NO_DEVICE	command-interface signature absent
2	ULTIMATE_ERR_TIMEOUT	polling budget exhausted
3	ULTIMATE_ERR_PROTOCOL	malformed or inconsistent transport re- ply
4	ULTIMATE_ERR_NOT_SUPPORTED	target, command or direct hardware unavailable
5	ULTIMATE_ERR_INVALID_ARGUMENT	caller supplied an invalid argument
6	ULTIMATE_ERR_IO	filesystem, network or transfer failure
7	ULTIMATE_ERR_DEVICE	target reported a failure
8	ULTIMATE_ERR_TRUNCATED	retained buffer did not hold the whole reply
9	ULTIMATE_ERR_ABORTED	exchange was abandoned
10	ULTIMATE_END	directory or socket stream ended nor- mally

ultimate_strerror() returns a printable description. ultimate_device_code() returns the last target's raw numeric status for diagnostics; its meaning depends on the target.

ONE LIBRARY, FOUR WAYS IN

Use files, REU memory, the live palette, sockets and HTTP from a C64 program without implementing the command-interface handshake.

BASIC, cc65 C, ca65 assembly and the standalone jump table all reach the same 6502 implementation.

- BASIC V2 keywords at the `READY.` prompt
- A cc65 library and public headers
- A documented ca65 calling convention
- A standalone jump table for other toolchains

```
READY.  
PRINT UREU  
32  
READY.
```

This expression reads the enabled REU size in 64K banks; 32 means 2MB.

FOR THE 1541 ULTIMATE-II, THE ULTIMATE 64 AND THE COMMODORE 64 ULTIMATE

FIRST EDITION · MIT LICENCE